Two-Machine Flow Shop Scheduling to Minimize Total Weighted Completion Time: Structural Properties and Dynamic Programming

Author One[a,*], Author Two[b]

[a]Institution One, Location, Country

[b]Institution Two, Location, Country

Abstract

Minimizing the total weighted completion time on a two-machine flow shop, denoted $F2 \parallel \sum w_j C_j$, is a classical NP-hard scheduling problem with applications in manufacturing, computing, and logistics. Existing exact methods rely on enumerative techniques that scale poorly with instance size, while approximation algorithms often impose restrictive assumptions on processing times. Here we partition the job set into at most K categories by non-increasing weight, which decomposes the schedule into K sequential blocks. We establish four structural properties of optimal schedules: the permutation property, the positional constraint $\pi(J_k) \leq k$, the block-end condition requiring the final job of each block to carry that block's weight class, and the SPT(1)-LPT(2) (Johnson) ordering rule within each block. Building on these properties and the interval-based dynamic programming framework developed for $F2|r_j|C_{\max}$, we design a pseudo-polynomial time dynamic programming algorithm that constructs the schedule block by block. We further show that geometric rounding of weights and processing times converts this algorithm into a polynomial-time approximation scheme.

Keywords: Flow shop scheduling; Total weighted completion time; Dynamic programming; Structural properties

*Corresponding author. E-mail address: author@email.com

1 Introduction

2 Problem Formulation

3 Structural Properties

We partition the job set J into K weight categories $J_1, J_2, \ldots, J_K$ such that all jobs in J_k share the same weight W_k, with $W_1 > W_2 > \cdots > W_K$. Let $n_k = |J_k|$ and $\sum_{k=1}^{K} n_k = n$.

The schedule is divided into K contiguous logical segments (blocks) $S_1, S_2, \ldots, S_K$, processed sequentially on both machines. We denote by $\pi(j)$ the block index to which job j is assigned.

Lemma 3.1 ***(Permutation property.)*** *For $F2 \parallel \sum w_j C_j$, there exists an optimal schedule in which the job sequence is identical on both machines.*

This is a classic result for two-machine flow shops with regular objective functions.

Lemma 3.2 ***(Positional constraint.)*** *In any optimal schedule, $\pi(J_k) \leq k$ for all $k = 1, \ldots, K$. Equivalently, every job from weight class J_k is assigned to one of the first k blocks $S_1, S_2, \ldots, S_k$.*

Lemma 3.2 implies that once block S_k has been processed, all jobs carrying weights $W_1, \ldots, W_k$ are fully scheduled. Consequently, block S_k may contain only jobs from classes $J_k, J_{k+1}, \ldots, J_K$.

Lemma 3.3 ***(Block-end condition.)*** *In any optimal schedule, the final job of block S_k belongs to weight class J_k, for each $k = 1, \ldots, K$.*

Lemma 3.4 ***(Intra-block ordering.)*** *Within each block S_k, all jobs are sequenced according to the SPT(1)-LPT(2) rule (Johnson's rule): for any two jobs $x, y \in S_k$, job x precedes job y if and only if*

$$\min\{a_x, b_y\} < \min\{a_y, b_x\},$$

with ties broken by non-decreasing a_j.

Taken together, these four properties determine the structure of any optimal schedule: jobs are organised into K blocks ordered by decreasing weight; within each block the sequence follows Johnson's rule; class k jobs cannot appear beyond block k; and the final position of block k is occupied by a job of class k.

4 Dynamic Programming Algorithm

The structural properties established in Section 3 enable a dynamic programming algorithm that constructs the schedule by iteratively inserting jobs into a partial permutation. Our approach adapts the interval-based state representation of , originally devised for the $F2|r_j|C_{\max}$, to the weighted completion time objective. In that framework, the schedule is partitioned into intervals, each interval is internally sequenced by Johnson's rule, and the DP state tracks per-interval completion times. We retain the Johnson ordering within blocks but replace the interval-based state space with a permutation-based representation and an incremental weighted sum computation.

4.1 Preprocessing

We first partition the n jobs into K sets $J_1, J_2, \ldots, J_K$ by weight, with $W_1 > W_2 > \cdots > W_K$. Within each set J_k, we sort the jobs according to Johnson's rule (SPT(1)-LPT(2)). Let the resulting ordered list be

$$L_k = \big(j_{k,1}, j_{k,2}, \ldots, j_{k,n_k}\big), \quad k = 1, 2, \ldots, K,$$

where $n_k = |J_k|$.

4.2 State Definition

A state is a 5-tuple (π, A, B, z, last_k), where

- π is the current partial permutation of already-scheduled jobs;
- A and B are the cumulative M1 and M2 completion times when processing π through the flow shop;
- z is the accumulated weighted sum of completion times for all jobs in π;
- last_k records the rightmost position in π occupied by a job of the current class J_k (0 if no job of J_k has been inserted yet).

The initial state is ($\emptyset$, 0, 0, 0, 0).

4.3 Dynamic programming Algorithm

The algorithm constructs an optimal schedule by inserting jobs in the pre-sorted order $L_1, L_2, \ldots, L_K$. At each step, a single job is inserted into every feasible position of the current

partial permutation, and the structural constraints established in Section 3 are enforced incrementally to prune infeasible prefixes.

State transition. Given a state (π, A, B, z, last_k) with $N = |\pi|$ and a job $j \in J_k$ to be inserted, the algorithm evaluates $N + 1$ candidate positions $\text{pos} \in \{0, 1, \ldots, N\}$. Position pos corresponds to inserting j after the first pos jobs of π, with $\text{pos} = 0$ denoting insertion at the front of the permutation.

Single-step expansion. For each candidate position pos:

1. **Construct** the augmented permutation $\pi' = \pi[1..\text{pos}] \parallel j \parallel \pi[\text{pos}{+}1..N]$.
2. **Update** $\text{last}'_k = \text{pos} + 1$.
3. **Compute** the cumulative metrics for π' by simulating the flow shop:

$$A' = \sum_{x \in \pi'} a_x,$$
$$B' = \text{flow-shop-M2-completion}(\pi'),$$
$$z' = \sum_{x \in \pi'} w_x \cdot C_x,$$

   where C_x denotes the completion time of job x on M2 under permutation π'.
4. **Validate** the block-end constraint. When j is the final job of J_k (i.e., all n_k jobs of class k have been processed), verify that $\text{last}'_k \geq k$. Candidates that violate this condition are discarded.

Correctness of early pruning. After the final job of class J_k has been inserted, the scalar last_k records its position in the current permutation. The check $\text{last}'_k \geq k$ enforces that the last job of class k occupies a position no earlier than k, which is necessary for the block structure to accommodate one job from each of the k weight classes preceding it. Should this condition be violated, subsequent insertions from classes $J_{k+1}, \ldots, J_K$ can only increase last_k (by shifting it rightward when jobs are inserted before position last_k), so the inequality $\text{last}_k \geq k$ may still be restored. However, states that already satisfy the condition are carried forward; those that do not are discarded early to limit state growth.

Positional constraint. The positional constraint of Lemma 3.2 is implicitly enforced by the insertion mechanism. Although a job $j \in J_k$ may be inserted at any position, the check $\text{last}'_k \geq k$ at each class boundary ensures that the final job of class k occupies a position no earlier than k, which combined with the block-end condition guarantees that no job of class k appears beyond block k in any surviving state.

Dominance-based pruning. For each parent state (π, A, B, z, last_k) and job j, the algorithm retains only those candidate positions that attain the minimum objective value.

Formally, let

$$z^* = \min_{\text{pos}\in\{0,\ldots,N\}} \{\, z'_{\text{pos}} \mid \text{candidate at pos is valid} \,\}.$$

All candidates achieving $z' = z^*$ are inserted into $\mathcal{Y}_{\text{new}}$; the remainder are discarded. This dominance rule is exact: for a fixed set of unscheduled jobs, the completion time contribution of any suffix schedule is monotone non-decreasing in the prefix completion times (A, B). A sub-optimal insertion of a single job therefore cannot be recovered by any subsequent extension of the partial schedule.

Termination. Upon processing all n jobs, every surviving state satisfies the four structural properties of Section 3 by construction. The optimal total weighted completion time is

$$Z^* = \min\{\, z \mid (\pi,\ A,\ B,\ z,\ \text{last}_k) \in \mathcal{Y} \,\}.$$

The complete procedure is given in Algorithm 1.

Algorithm 1 DP_F2_WeightedSum

Require: Jobs partitioned into $J_1, \ldots, J_K$ with $W_1 > \cdots > W_K$; each L_k is the Johnson-ordered list of J_k.
Ensure: optimal weighted sum Z^*

$\mathcal{Y} \leftarrow \{(\emptyset,\ 0,\ 0,\ 0,\ 0)\}$
for $k = 1$ to K **do**
  for $i = 1$ to n_k **do**
    $j \leftarrow L_k[i]$
    $\mathcal{Y}_{\text{new}} \leftarrow \emptyset$
    for each $(\pi,\ A,\ B,\ z,\ \text{last}_k) \in \mathcal{Y}$ **do**
      $N \leftarrow |\pi|$
      $\text{best}_z \leftarrow +\infty$; best_states $\leftarrow \emptyset$
      for pos $= 0$ to N **do**
        $\pi' \leftarrow \pi[1..\text{pos}] \parallel j \parallel \pi[\text{pos}{+}1..N]$
        $\text{last}'_k \leftarrow \text{pos} + 1$
        Simulate π' through the flow shop to compute (A', B', z')
        if $i = n_k$ **then**
          check $\text{last}'_k \geq k$
          if violated **then continue**
          end if
        end if
        if $z' < \text{best}_z$ **then**
          $\text{best}_z \leftarrow z'$; best_states $\leftarrow \{(\pi', A', B', z', \text{last}'_k)\}$
        else if $z' = \text{best}_z$ **then**

21: best_states $\leftarrow$ best_states $\cup \{(\pi', A', B', z', \text{last}'_k)\}$
22: **end if**
23: **end for**
24: $\mathcal{Y}_{\text{new}} \leftarrow \mathcal{Y}_{\text{new}} \cup$ best_states
25: **end for**
26: $\mathcal{Y} \leftarrow \mathcal{Y}_{\text{new}}$
27: **end for**
28: **end for**
29: $Z^* \leftarrow \min\{\, z \mid (\pi,\, A,\, B,\, z,\, \text{last}_k) \in \mathcal{Y} \,\}$
30: **return** Z^*

4.4 Complexity Analysis

Let $|\mathcal{Y}|_{\max}$ denote the maximum number of states retained in any single iteration. For each state with prefix length N, the algorithm evaluates $N+1$ insertion positions, and each simulation of π' takes $O(N)$ time, yielding $O(N^2)$ work per state per job. Since N grows from 0 to $n-1$ as jobs are inserted, the total time is

$$O\big(|\mathcal{Y}|_{\max} \cdot n^3\big).$$

The state retention rule (keeping only minimal-z candidates from each parent state) keeps $|\mathcal{Y}|_{\max}$ small in practice. The $\text{last}_k \geq k$ checks at class boundaries further prune infeasible states early, preventing exponential blow-up.

5 Conclusion